

MATLAB SESSION 2

Scripts, Loops, Functions and Plots

Dr Sumanth Bharadwaj H S
Department of Mathematics and Statistics
M S Ramaiah University of Applied Sciences.

Topics Covered

1. Scripts

- ▶ Writing and running a script
- ▶ Comments and sections
- ▶ Formatted output

2. Decisions and loops

- ▶ Branching with `if`
- ▶ `for` and `while` loops
- ▶ Stopping on a tolerance

3. Functions

- ▶ Writing a function file
- ▶ Outputs and arguments
- ▶ Function handles

4. Plotting

- ▶ Drawing and labelling a curve
- ▶ Several curves on one axes
- ▶ Data points and model lines

Session outcomes

On completion of this session the student will be able to

1. Write, save and run a script file, and report its results in a readable form using `fprintf`.
2. Construct a program using decisions and loops, stopping on a tolerance rather than a fixed count, and guarded by an iteration limit.
3. Write a function with arguments, several outputs and default values, and pass a function to it as a handle so that one method serves any problem of its type.
4. Produce a labelled two-dimensional plot that presents computed results and measured data appropriately.

MATLAB SESSION 2

Scripts

Keeping your work

Writing and running a script

Commands typed at the prompt are lost. A script is a file of commands that runs from top to bottom, so you can correct one line and run the whole thing again.

- ▶ Create one with **Home**, then **New**, then **Script**.
- ▶ Save it with a name that starts with a letter and has no spaces, for example `newton_test.m`.
- ▶ Run it with **Run**, or by typing its name without the extension.

```
% newton_test.m
% Solves a small system and reports the residual.

A = [2 1 -1; -3 -1 2; -2 1 2];
b = [8; -11; -3];

x = A\b;
r = max(abs(A*x - b));    % largest error in the solution
```

Writing and running a script (continued)

The file name is how MATLAB finds the script, and it must be in the Current Folder. A script called `sum.m` will hide the built-in `sum`, so avoid naming files after functions.

Type this now. Create the script above, save it, and run it. Then check the Workspace: `A`, `b`, `x` and `r` should all be there, because a script shares the same workspace as the Command Window.

Comments and sections

A percent sign starts a comment. Use comments to record what a quantity means and what units it is in, not to restate the syntax.

```
tol = 1e-10;      % stop when the step is smaller than this
maxIter = 50;     % give up after this many iterations
```

Two percent signs start a **section**. Sections divide a long script into parts you can run separately with **Ctrl+Enter**, which is how you develop a program: write a few lines, run only those, look at the result.

```
%% Set up
A = [4 -2 1; -2 4 -2; 1 -2 4];
b = [11; -16; 17];

%% Solve and check
x = A\b;
residual = max(abs(A*x - b))
```

Comments and sections (continued)

Type this now. Split your script into two sections as above and run each with Ctrl+Enter.

Printing results

Leaving off a semicolon shows a value with its variable name. For anything more controlled, use `fprintf`.

```
>> x = 2.0945515;  
>> fprintf('root = %.6f\n', x)  
root = 2.094552
```

Code	Meaning
%d	Integer
%f	Decimal; %.6f gives six decimal places
%e	Scientific notation, for very small or large values
%g	Whichever of the two is shorter
\n	New line. Without it everything runs together.

Printing results (continued)

Given a vector, `fprintf` repeats the format until the values run out, which prints a whole column of results in one command.

```
>> fprintf('%6.3f\n', [1.5 2.25 3.125])  
1.500  
2.250  
3.125
```

Two values per line are produced by putting two codes in the format, and MATLAB reads down the columns of a matrix as it fills them.

`disp(x)` is the simpler alternative when no formatting is needed: it shows the value alone, with no variable name and no blank lines.

MATLAB SESSION 2

Decisions

Comparisons and if

Comparisons

A comparison produces true or false, shown as 1 or 0.

Operator	Meaning	Note
==	equal to	Two signs. One sign assigns.
~=	not equal to	
< >	less, greater	
<= >=	at most, at least	Use inside if
&&	and	
	or	
~	not	

Comparisons (continued)

Never compare computed decimals with `==`. Compare the difference against a tolerance: `abs(a - b) < tol`. This is the single most important line in every iterative program you will write.

Conditional statements: if, elseif, else

Syntax:

```
if condition
    statements
elseif condition
    statements
else
    statements
end
```

For example:

```
if d == 0
    error('Division by zero.')
elseif abs(d) < 1e-12
    warning('Divisor is very small; result may be unreliable.')
else
    q = n/d;
end
```

if, elseif, else (continued)

- ▶ Only the first branch whose condition is true runs.
- ▶ The `elseif` and `else` parts are optional, and `end` is required.
- ▶ `error` stops the program and prints your message.
- ▶ `warning` prints and carries on.
- ▶ Both take the same formatting codes as `fprintf`, so a message can include the offending value.

```
error('Derivative is zero at x = %g.', x)
```

if: Examples

Classifying the roots of $ax^2 + bx + c = 0$ by the sign of the discriminant.

```
a = 1; b = -5; c = 6;  
d = b^2 - 4*a*c;  
  
if d > 0  
    fprintf('Real and distinct roots\n')  
elseif d == 0  
    fprintf('Real and equal roots\n')  
else  
    fprintf('Complex conjugate roots\n')  
end
```

if: Examples (continued)

Checking that a matrix is usable before solving with it.

```
A = [4 -2; 1 3];  
b = [2; 11];  
  
if abs(det(A)) < 1e-12  
    error('Matrix is singular to working precision.')  
else  
    x = A\b;  
    fprintf('x1 = %g, x2 = %g\n', x(1), x(2))  
end
```

The test uses a tolerance rather than $\det(A) == 0$, for the reason given earlier: a determinant computed in floating point is rarely exactly zero.

if, elseif, else (continued)

A program that stops with a clear message is far better than one that returns a wrong answer quietly. When you write your own methods, check for the ways they can fail and stop deliberately.

Type this now. Write a code to classify a stationary point of a function as a local minimum, a local maximum, or an inconclusive test based on the sign of the second derivative $d = f''(c)$.

MATLAB SESSION 2

Repetition

for and while

for: a known number of repetitions

Syntax:

```
for variable = vector
    statements
end
```

The loop variable takes each value of the vector in turn, and `end` closes the loop.

For example:

```
total = 0;
for k = 1:5
    total = total + k^2;
end
fprintf('sum of squares = %d\n', total);
```

The loop variable takes each value of the vector in turn. Any vector works, not only a range: `for k = [2 3 5 7]` runs four times.

for: preallocating the result

When a loop fills an array, create the array at full size first. Growing it one entry at a time makes MATLAB rebuild it on every pass.

```
x = zeros(1, 20);           % reserve the space first
for k = 1:20
    x(k) = 1/k;
end
```

for: Examples

Summing a series. Here $\sum_{k=1}^{20} 1/k^2$, which approaches $\pi^2/6$.

```
n = 20;
s = 0;
for k = 1:n
    s = s + 1/k^2;
end
fprintf('sum = %.6f, pi^2/6 = %.6f\n', s, pi^2/6)
```

Each element of k here is independent of the others, so `s = sum(1 ./ (1:n).^2);` does the same work of for loop in one line. The loop is shown for comparison.

for: Examples (continued)

Building a table of values, with the array reserved in advance.

```
x = linspace(0, 1, 11);  
y = zeros(size(x));          % reserve the space first  
  
for k = 1:numel(x)  
    y(k) = exp(-x(k)) * cos(4*pi*x(k));  
end  
  
fprintf('%6.2f %10.5f\n', [x; y])
```

Each element of y here is independent of the others, so $y = \exp(-x) .* \cos(4\pi x)$ does the same work in one line. The loop is shown for comparison.

for: a known number of repetitions (continued)

Type this now. Write a loop that prints the first eight terms of $2^{1/k}$ using `fprintf`, one per line.

while: repeat until a condition is met

Use `while` when you do not know in advance how many repetitions are needed, which is the case for every iterative method. Syntax:

```
while condition
    statements
end
```

For example:

```
x    = 1;
step = Inf;
tol  = 1e-10;

while abs(step) > tol
    step = (x - 2/x)/2;
    x    = x - step;
end
```

while: repeat until a condition is met (continued)

Two things must be true or the loop never ends: the variable in the condition must have a value before the loop starts, and something inside must change it.

A **while** loop that fails to converge runs forever. Always add a counter and a limit. Press **Ctrl+C** in the Command Window to interrupt a loop that has hung.

```
iter = 0;
while abs(step) > tol
    iter = iter + 1;
    if iter > 100
        error('No convergence in %d iterations.', iter)
    end
    % ... the step ...
end
```

while: Examples

Bisection on $f(x) = x^3 - 2x - 5$, halving the bracket until it is narrow enough.

```
f = @(x) x.^3 - 2*x - 5; %This will be explained later
a = 2; b = 3; tol = 1e-8;

while (b - a)/2 > tol
    c = (a + b)/2;
    if f(a)*f(c) <= 0
        b = c;
    else
        a = c;
    end
end
fprintf('root = %.8f\n', (a+b)/2)
```

while: Examples (continued)

The smallest n for which $n!$ exceeds a million. The count is not known in advance, so `while` is the natural choice.

```
n = 1;
while factorial(n) <= 1e6
    n = n + 1;
end
fprintf('n = %d, n! = %d\n', n, factorial(n))
```

Both loops change the quantity appearing in their own condition: the bracket narrows, and n grows. A `while` whose condition never changes is the commonest way to hang MATLAB.

Choosing between for and while

Use for when	Use while when
The count is known before you start Filling an array of known size Trying each of a fixed list of values	The count depends on the answer Iterating until an error is small enough Refining a guess

A `for` loop with a `break` inside can do the work of a `while` loop, and has the advantage that the iteration limit is built into the range. Many of the methods you write later will use that pattern.

```
for iter = 1:maxIter
    % ... compute the step ...
    if abs(step) < tol
        break           % leave the loop early
    end
end
```

MATLAB SESSION 2

Functions

Writing a method once

A function file

A function takes inputs, returns outputs, and keeps its own private set of variables. It is how you write a method once and use it on any problem. Syntax:

```
function [out1, out2] = name(in1, in2)
    statements
end
```

For example:

```
function r = residual(A, x, b)
%RESIDUAL Size of the error in a claimed solution of A*x = b.
    r = norm(A*x - b);
end
```

Save this as `residual.m`. The file name must match the function name.

A function file (continued)

```
>> residual(A, x, b)
ans =
    1.2561e-15
```

The comment lines directly under the first line are the help text. Typing `help residual` prints them, so write them.

Variables inside a function are invisible outside it, and the workspace variables are invisible inside. Everything the function needs must be passed in as an argument. This is a feature: it means a function cannot be broken by whatever else happens to be in your workspace.

Functions: Examples

The roots of a quadratic, returned as two outputs.

```
function [r1, r2] = quadroots(a, b, c)
%QUADROOTS Roots of a*x^2 + b*x + c = 0.
    d = sqrt(b^2 - 4*a*c); % complex if the discriminant is negative
    r1 = (-b + d)/(2*a);
    r2 = (-b - d)/(2*a);
end
```

```
>> [p, q] = quadroots(1, -5, 6)
p =
    3
q =
    2
```

Functions: Examples (continued)

The trapezoidal rule, written once and usable on any function.

```
function I = trap(f, a, b, n)
%TRAP  Integral of f from a to b, using n strips.
    h = (b - a)/n;
    x = a:h:b;
    y = f(x);
    I = h*(y(1) + 2*sum(y(2:end-1)) + y(end))/2;
end
```

```
>> trap(@(x) sin(x), 0, pi, 100)
ans =
    1.9998
```

The exact value is 2. Because `f` arrives as a handle, the same function integrates any integrand.

Several outputs, and optional arguments

Syntax:

```
function [out1, out2] = name(in1, in2, in3)
    statements
end
```

For example:

```
function [root, iter] = mymethod(f, x0, tol, maxIter)

    if nargin < 3, tol      = 1e-10; end
    if nargin < 4, maxIter = 50;    end

    % ... the method ...
end
```

Outputs are listed in square brackets and supplied left to right.

Several outputs (continued)

Asking for fewer outputs than are offered is allowed.

```
>> r = mymethod(f, 2);           % just the root  
>> [r, n] = mymethod(f, 2);      % the root and the count
```

`nargin` is the number of arguments the caller actually supplied, so the two `if` lines give `tol` and `maxIter` sensible defaults when they are omitted. Every method you write should have defaults for its tolerance and its iteration limit.

Type this now. Write a function `stats(v)` returning the mean, the smallest and the largest value of a vector, and call it three ways: asking for one output, then two, then all three.

Function handles

To write a method that works on any equation, you must be able to pass the equation to it. A function handle is a function stored in a variable.

Syntax:

```
name = @(arguments) expression
```

For example:

```
>> f = @(x) x.^3 - 2*x - 5;
>> f(2)
ans =
    -1

>> f([1 2 3])
ans =
    -6    -1    16
```

The part in brackets names the input; the rest is the formula. Note the dot operators, which let the same handle accept a whole vector of values at once.

Function handles (continued)

A handle can be passed into a function like any other value, which is what makes a general-purpose method possible.

```
function r = mymethod(f, df, x0)
    % f and df arrive as handles and are called normally
    step = f(x0)/df(x0);
end
```

```
>> mymethod(@(x) x.^2 - 2, @(x) 2*x, 1)
```

A handle captures the values of any other variables at the moment it is created. Writing `a = 2; g = @(x) a*x;` and then changing `a` does not change `g`.

Local functions in a script

A short helper can live at the bottom of the script that uses it, rather than in a file of its own. All functions must come after every line of script code.

```
% --- script code ---
x = bisect(@(x) x.^2 - 2, 1, 2);
fprintf('root = %.8f\n', x);

% --- functions ---
function m = bisect(f, a, b)
    m = (a + b)/2;      % one step only, for illustration
end
```

This keeps a piece of lab work in a single file, which is convenient for submission. For anything you will reuse across topics, give the function its own file instead.

When something goes wrong

Message	Usual cause
Unrecognized function or variable	File name does not match, or wrong Current Folder
Too many output arguments	Asking for [a,b] from a function returning one
The program never finishes	A <code>while</code> whose condition never becomes false
Nothing is displayed	A semicolon on the line you wanted to see

Using a breakpoint

Click the dash to the left of a line number in the Editor to set a **breakpoint**. Run the program and it stops just before that line, with every variable available to inspect. This is far quicker than rereading the code, and it is the tool to reach for when a method will not converge.

Type this now. Put a breakpoint inside the `while` loop from earlier, run it, and inspect `x` and step after a few iterations.

What you should be able to do now

- ▶ Write, save and run a script, and split it into sections.
- ▶ Print a formatted table of results with `fprintf`.
- ▶ Use `if`, `elseif` and `else`, and stop a program with error.
- ▶ Write a `for` loop, and preallocate an array before filling it.
- ▶ Write a `while` loop with a tolerance and an iteration limit.
- ▶ Write a function with several outputs and default arguments.
- ▶ Create a function handle and pass it into another function.
- ▶ Set a breakpoint and inspect variables while a program is stopped.

MATLAB SESSION 2

Plotting

Seeing the answer, not just reading it

Syntax of a MATLAB plot

1. Choose the x values, usually with `linspace`.
2. Compute the y values, using element-wise operators.
3. Call `plot`, then label the result.

Syntax:

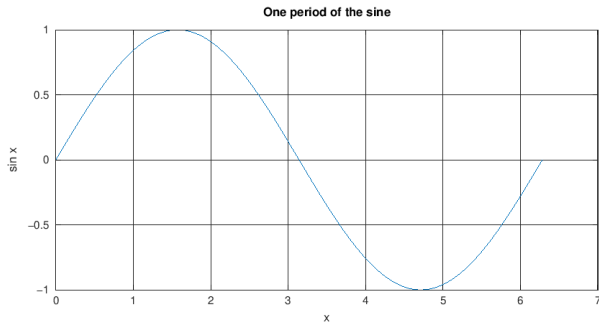
```
plot(x, y)           % x and y must have the same length  
plot(x, y, 'style')  % colour, line style and marker
```

plot: Example

```
x = linspace(0, 2*pi, 200);  
y = sin(x);  
  
plot(x, y)  
grid on  
xlabel('x')  
ylabel('sin x')  
title('One period of the sine')
```

`linspace(a, b, n)` gives `n` values evenly spaced from `a` to `b`, both endpoints included. Around 100 to 300 points gives a smooth curve; with 10 the result looks like a polygon, because `plot` draws straight lines between the points you supply.

Three steps, every time (continued)

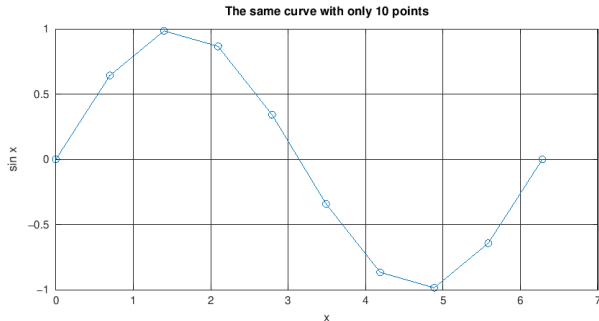


The result of the commands above.

Three steps, every time (continued)

Type this now. Draw the curve above. Then repeat with 10 points instead of 200 and compare.

Three steps, every time (continued)



The same curve with only 10 points: plot joins your samples with straight lines.

The two errors

Error 1:

```
>> x = linspace(0, 1, 50);  
>> y = linspace(0, 1, 40);  
>> plot(x, y)  
Error using plot  
Vectors must be the same length.
```

The x and y arrays must have the same number of entries. This happens when you change the number of points in one line and forget to re-run the other. Check with `numel(x)` and `numel(y)`.

The two errors

Error 2:

```
>> x = linspace(0, 2, 100);  
>> y = x^2;  
Error using ^  
Incorrect dimensions for raising a matrix to a power.
```

A missing dot. Computing y at many x values at once is element-wise, so it must be $x.^2$. The same applies to $.*$ and $./$.

This is the most common plotting mistake in the course. If you are evaluating a formula at a vector of points, every multiplication, division and power in it needs a dot.

Type this now. Plot $y = x^2 e^{-x}$ for $0 \leq x \leq 6$. It needs two dot operators; find both.

MATLAB SESSION 2

Making the plot readable

Labels, grid and legend

Command	Effect
<code>xlabel('...')</code>	Label on the horizontal axis
<code>ylabel('...')</code>	Label on the vertical axis
<code>title('...')</code>	Title above the axes
<code>grid on</code>	Adds a grid, making values easier to read off
<code>legend('a','b')</code>	Names the curves, in the order they were drawn
<code>axis equal</code>	Equal scale on both axes
<code>xlim([0 5])</code>	Sets the visible range; also <code>ylim</code>

Put units in the axis labels whenever the quantity has them. A figure should make sense without you standing next to it explaining it.

Labels, grid and legend (continued)

A backslash in a label gives Greek letters and mathematics: `xlabel('\theta')` prints the Greek theta, and `title('x^2')` raises the 2. This works in any label or title.

Line style, colour and markers

A third argument to `plot` is a short string setting the colour, the line style and the marker in any order.

```
plot(x, y, 'r')      % red solid line
plot(x, y, 'b--')    % blue dashed
plot(x, y, 'ko')     % black circles, no line
plot(x, y, 'g-.s')   % green dash-dot with square markers
```

Group	Codes
Colours	r red, g green, b blue, k black, m magenta, c cyan, y yellow
Line styles	- solid, -- dashed, : dotted, - . dash-dot
Markers	o circle, s square, d diamond, * star, + plus, x cross, ^ triangle

Giving a marker with no line style plots the points alone, with nothing joining them. That distinction matters in the next frame.

Line style, colour and markers (continued)

For thicker lines or larger markers, add a name and a value after the string:

```
plot(x, y, 'r-', 'LineWidth', 2)
plot(xd, yd, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k')
```

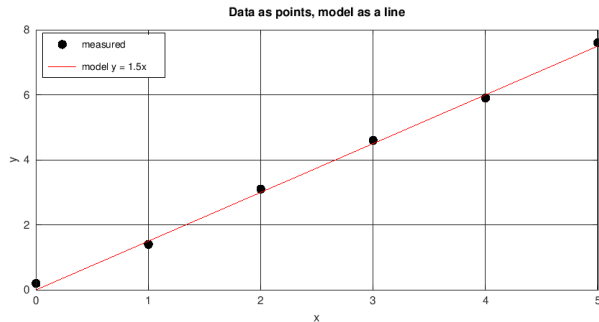
Data as points, curves as lines

Measured values are separate observations; you did not measure anything between them. Draw them as markers with no connecting line. A formula or a fitted model is defined everywhere, so draw it as a continuous line.

```
xd = [0 1 2 3 4 5];           % measured points
yd = [0.1 0.9 2.1 2.8 4.2 5.1];

plot(xd, yd, 'ko')             % data: markers only
hold on
xf = linspace(0, 5, 200);
plot(xf, xf, 'r-')             % model: a line
hold off
legend('measured', 'model')
```

Data as points, curves as lines (continued)



Measurements as markers; the model as a continuous line.

Data as points, curves as lines (continued)

Following this convention makes a figure self-explanatory: the reader sees at once which quantities were observed and which were computed.

Several curves on one axes

`plot` clears the axes before drawing. To add to an existing figure, use `hold on`.

```
x = linspace(-2*pi, 2*pi, 300);  
  
plot(x, sin(x), 'b-')  
hold on  
plot(x, cos(x), 'r--')  
hold off  
  
legend('sin x', 'cos x')  
grid on
```

An alternative for curves you already have is to pass several pairs to one call:
`plot(x, y1, x, y2)`.

Forgetting `hold off` means the next figure you draw is added to this one. If a plot mysteriously contains an old curve, this is why.

Several curves on one axes (continued)

`figure` opens a new, empty window, so a script that draws several separate graphs should call it before each.

Several curves on one axes (continued)



Two curves, distinguished by style and named in the legend.

Several curves on one axes (continued)

Type this now. On one axes, draw $\sin x$ and $\sin 2x$ for $0 \leq x \leq 2\pi$, in different styles, with a legend and a grid.

MATLAB SESSION 2

Two more plotting features

Plotting a function directly

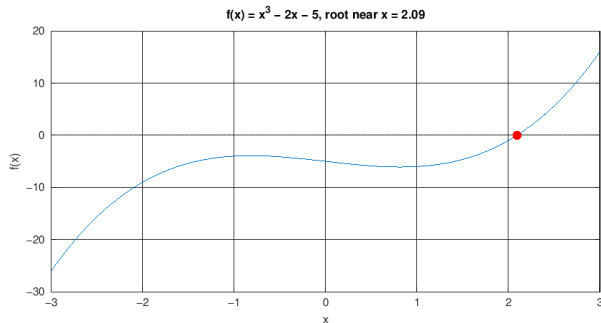
`fplot` takes a function handle and chooses its own x values, adding more where the curve bends sharply.

```
f = @(x) x.^3 - 2*x - 5;  
  
fplot(f, [-3 3])  
grid on  
ylines(0)           % a horizontal reference line at y = 0
```

This is a quick way to see roughly where a curve crosses the axis, which is exactly what you need when choosing a starting guess for an iterative method. `ylines(0)` draws a horizontal line, and `xlines(2)` a vertical one. Both accept a style and a label:

```
xlines(2.0946, 'r--', 'root')
```

Plotting a function directly (continued)



Reading a starting guess for an iterative method straight off the graph.

Plotting a function directly (continued)

Type this now. Use `fplot` to draw $x^3 - 2x - 5$ on $[-3, 3]$ with a line at $y = 0$, and read off roughly where it crosses.

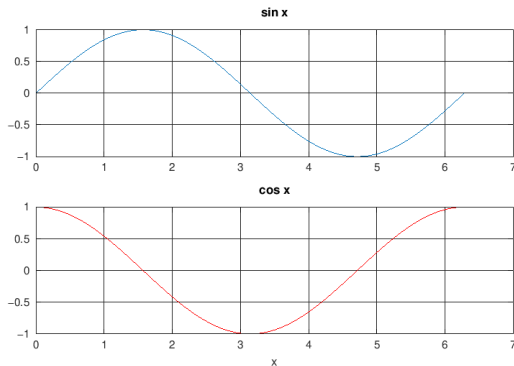
Several plots in one figure

`subplot(m, n, k)` divides the figure into an m by n grid and selects panel k , numbered along the rows.

```
x = linspace(0, 2*pi, 200);  
  
subplot(2, 1, 1)  
plot(x, sin(x)); title('sin x'); grid on  
  
subplot(2, 1, 2)  
plot(x, cos(x)); title('cos x'); grid on
```

Every plotting command after a `subplot` call applies to that panel only, so each needs its own labels.

Several plots in one figure (continued)



Two panels, each with its own title and grid.

Several plots in one figure (continued)

Newer MATLAB also has `tiledlayout(2,1)` with `nexttile`, which spaces panels more tightly and needs no index arithmetic. Both are current; `subplot` is what you will meet in most existing code.

Saving a figure

Command	Result
<code>saveas(gcf, 'plot.png')</code>	Image file, for a report
<code>exportgraphics(gcf, 'p.png', 'Resolution', 300)</code>	Higher quality, cropped
<code>exportgraphics(gcf, 'p.pdf')</code>	Vector output, sharp at any zoom

`gcf` means the current figure. For a lab report, export at 300 resolution or as a PDF rather than taking a screenshot of the window, which comes out blurred in print.

What you should be able to do now

- ▶ Draw a function using `linspace`, element-wise operators and `plot`.
- ▶ Diagnose the two standard errors: unequal lengths, and a missing dot.
- ▶ Label a figure with axis names, a title, a grid and a legend.
- ▶ Choose colour, line style and markers, and set the line width.
- ▶ Draw data as markers and a model as a line on the same axes.
- ▶ Add curves with `hold on`, and start a fresh window with `figure`.
- ▶ Use `fplot` for a quick look at a function, with `xline` and `yline` for reference.
- ▶ Place several panels with `subplot`, and export a figure for a report.

Problems

1. Write a script that stores a three-by-three matrix and a right-hand side, solves the system, and prints the solution and the residual with `fprintf` to eight decimal places.
2. Write a `for` loop that computes $\sum_{k=1}^{100} 1/k^2$ and prints a running total every twenty terms. Preallocate any array you use.
3. Write a `while` loop that starts at $x = 1$ and repeatedly replaces x by $(x + 3/x)/2$ until two successive values differ by less than $1e-12$. Print the number of iterations and the final value. Add a limit of 100 iterations with an error if it is exceeded.
4. Write a function `stats(v)` returning the mean, minimum, maximum and range of a vector, with help text. Demonstrate calling it with one, two and four outputs.
5. Write a function `applyTwice(f, x)` that returns $f(f(x))$. Test it with $@(x) x.^2$ and $@(x) x + 3$, and predict each answer before running it.

Problems (continued)

6. The function below is meant to count how many entries of a vector are positive. It runs and gives the wrong answer. Set a breakpoint, find the fault, and state it in one sentence.

```
function n = countPositive(v)
    n = 0;
    for k = 1:numel(v)
        if v(k) > 0
            n = 1;
        end
    end
end
```

7. Write a function that takes a handle f , a starting value a , a step h and a count n , and returns a column of f evaluated at a , $a+h$, up to $a+n*h$. Do it once with a loop and once without, and check the two agree with `isequal`.

Problems (continued)

8. Plot each function on a sensible range, fully labelled with a grid. State which dot operators each one needs.

▶ $y = x^3 - 6x^2 + 9x$ on $[-1, 5]$

▶ $y = e^{-x/2} \sin 4x$ on $[0, 8]$

▶ $y = \frac{1}{1 + x^2}$ on $[-10, 10]$

9. Draw $\sin x$ on $[0, 2\pi]$ four times in one figure, using `subplot(2,2,k)`, with 5, 10, 25 and 200 points. Title each panel with its point count. At what number of points does the curve stop looking like a polygon?

10. The data below are measurements. Plot them as black circles with no joining line, then add the straight line $y = 1.5x$ in red on the same axes, with a legend.

```
xd = [0 1 2 3 4 5];  
yd = [0.2 1.4 3.1 4.6 5.9 7.6];
```

Problems (continued)

11. Use `fplot` to draw $f(x) = \cos x - x$ on $[-2, 2]$ with a line at $y = 0$. Read off an approximate root, then confirm it with `fzero(f, 1)`.
12. Draw the circle $x = \cos t$, $y = \sin t$ for $0 \leq t \leq 2\pi$. It will look like an ellipse until you add one command. State which command and why.
13. Write a script that plots $\sin(kx)$ for $k = 1, 2, 3$ on one axes using a `for` loop and `hold on`, building the legend entries as it goes. Export the result as a 300-resolution PNG.